



Facultad de Ingeniería
Departamento de Ingeniería en Computación e Informática

Análisis e interpretación: Algoritmos recursivos de búsqueda y ordenamiento

Ayrton Morrison
Iván Gallardo
Pablo Gómez
Ingeniería Civil en Computación e Informática

Docente: MSc. Jackeline Aldridge
Ramo: Diseño de Algoritmos

RESUMEN

Este informe presenta el desarrollo de un sistema en lenguaje C para la gestión de información de deportistas, capaz de generar, almacenar, ordenar y buscar registros en archivos CSV. El proyecto se basa en la estrategia de *Divide y Vencerás*, implementando y analizando distintos algoritmos de ordenamiento, búsqueda y selección.

En particular, se implementaron los algoritmos de ordenamiento *Merge Sort* en su versión clásica y optimizada mediante *Insertion Sort*, además de *Quick Sort* utilizando el esquema de partición de Lomuto con distintas variantes de pivote: último elemento, primer elemento, pivote aleatorio y mediana de tres. Asimismo, se desarrollaron algoritmos de búsqueda como búsqueda binaria recursiva, búsqueda exponencial y búsqueda por interpolación, junto con una variante de búsqueda binaria para rangos. También se implementó el algoritmo de selección *Quick Select* para obtener el k-ésimo mejor deportista según puntaje.

El sistema incorpora funcionalidades prácticas como ordenamiento de deportistas según distintos criterios, búsqueda eficiente de registros, generación de rankings y obtención del mejor deportista en una posición determinada. Además, se realizaron benchmarks experimentales para evaluar el comportamiento de los algoritmos en distintos tamaños de entrada y escenarios de ejecución, considerando mejor caso, peor caso y caso promedio. Los resultados obtenidos permitieron comparar empíricamente el rendimiento de las distintas implementaciones y variantes estudiadas, contrastando además su comportamiento con el análisis teórico de complejidad temporal.

ÍNDICE GENERAL

Índice de figuras	v
1. Introducción	1
2. Objetivos	3
2.1. Objetivo Principal	3
2.2. Objetivos Secundarios	3
3. Marco teórico	5
3.1. Algoritmos de Ordenamiento	5
3.2. Algoritmos de Búsqueda	6
3.3. Algoritmos de Selección	6
4. Desarrollo	8
4.1. Diseño general del sistema	8
4.1.1. Estructura de datos de los deportistas	8
4.1.2. Generación y carga de datos	8
4.1.3. Funcionalidades del sistema	9
4.2. Algoritmos de ordenamiento	9
4.2.1. Merge Sort	9
4.2.1.1. Implementacion Merge Sort	11
4.2.2. Merge-Insertion Sort	12
4.2.2.1. Implementacion Merge-Insertion Sort	12
4.2.3. Quick Sort	12
4.2.3.1. Implementacion Quick Sort	13
4.3. Algoritmos de búsqueda	13
4.3.1. Búsqueda binaria recursiva	13
4.3.1.1. Implementacion Busqueda Binaria	14
4.3.2. Búsqueda exponencial	14
4.3.2.1. Implementacion Busqueda Exponencial	15
4.3.3. Búsqueda por interpolación	15
4.3.3.1. Implementacion Busqueda por Interpolacion	16
4.3.4. Búsqueda Binaria por Rangos	16
4.3.4.1. Implementacion Búsqueda Binaria por Rangos	17
4.4. Algoritmos de selección	17
4.4.1. Quick Select	17
4.4.1.1. Implementacion Quick Select	18
4.5. Benchmarks, Generacion de resultados y graficos	19
4.5.1. Generacion de graficos	19
4.6. Análisis de Resultados	19
4.6.1. Comparación de algoritmos de ordenamiento	20

4.6.1.1.	Análisis de mejor caso	20
4.6.1.2.	Análisis de caso promedio	22
4.6.1.3.	Análisis de peor caso	23
4.6.1.4.	Analisis Merge-Insertion Sort	24
4.6.1.5.	Análisis de peor caso	25
4.6.1.6.	Análisis de caso promedio	26
4.6.1.7.	Análisis Búsqueda Binaria por Rangos	28
4.6.2.	Análisis de Quick Select	28
4.6.2.1.	Análisis de mejor caso	29
4.6.2.2.	Análisis de caso promedio	29
4.6.2.3.	Análisis de peor caso	30
4.6.3.	Comparacion con algoritmos iterativos	30
4.6.3.1.	Comparacion de algoritmos de Búsquedas	31
4.6.3.2.	Comparacion de algoritmos de Ordenamiento	31
4.6.4.	Análisis General	33
5.	Conclusión	34
6.	Anexos	36
6.1.	Código fuente	36
6.2.	Contribución por integrante	36
6.2.1.	Iván Gallardo	36
6.2.2.	Pablo Gómez	36
6.2.3.	Ayrton Morrison	36

ÍNDICE DE FIGURAS

4.1. Benchmark de algoritmos de ordenamiento en mejor caso.	20
4.2. Benchmark de algoritmos de ordenamiento en caso promedio.	22
4.3. Benchmark de algoritmos de ordenamiento en peor caso.	23
4.4. Evaluación de Merge-Insertion: comparación de thresholds en escala lineal y logarítmica. . . .	24
4.5. Benchmark de algoritmos de búsqueda en peor caso.	25
4.6. Benchmark de algoritmos de búsqueda en caso promedio.	26
4.7. Benchmark de búsqueda binaria por rangos.	28
4.8. Benchmark de Quick Select en mejor caso.	29
4.9. Benchmark de Quick Select en caso promedio.	29
4.10. Benchmark de Quick Select en peor caso.	30
4.11. Benchmark de algoritmos de búsqueda: comparativa iterativos vs recursivos.	31
4.12. Benchmarks de ordenamiento secuencial: mejor, promedio y peor caso.	32
4.13. Benchmarks de ordenamiento recursivo: mejor, promedio y peor caso.	32

INTRODUCCIÓN

"**Divide y Vencerás**" es un paradigma de programación, que consiste en un conjunto de técnicas algorítmicas en las que primero se divide el problema a resolver en subproblemas de menor tamaño y de la misma naturaleza, luego se resuelven los subproblemas de manera independiente y recursiva, hasta llegar a un caso base (que es un caso tan trivial que detiene la recursión), y finalmente se combinan los resultados para llegar a la solución final.

En el presente informe, se analizará empíricamente la complejidad algorítmica de varios algoritmos que utilizan este paradigma de programación, y además, se evaluará el impacto que tiene cada variante de cada algoritmo.

Los algoritmos que serán objeto de este análisis involucrarán tanto algoritmos de ordenamiento, como algoritmos de búsqueda, como también un algoritmo de selección.

Estos algoritmos son:

Ordenamiento

- *Merge Sort*.
- *Merge-Insertion Sort* (versión de *Merge Sort* que incluye *Insertion Sort* cuando se llegan a tener subarreglos de tamaño pequeño).
- *Quick Sort* con pivote en la primera posición.
- *Quick Sort* con pivote en la última posición.
- *Quick Sort* con pivote en una posición aleatoria.
- *Quick Sort* escogiendo al pivote como resultado de una mediana entre 3 elementos.

Búsqueda

- Búsqueda binaria clásica.
- Búsqueda binaria para rangos.
- Búsqueda exponencial.
- Búsqueda por interpolación.

Selección

- *Quick Select*.

Todo lo anterior se encuentra enmarcado en un sistema de gestión de deportistas desarrollado previamente, el cual trabaja con registros que contienen información como ID, nombre, equipo, puntaje y cantidad de competencias disputadas. Sobre este conjunto de datos, se implementaron distintas funcionalidades prácticas, tales como el ordenamiento de deportistas según distintos criterios, la búsqueda eficiente de registros, la obtención del k-ésimo mejor deportista mediante algoritmos de selección, y la generación de rankings basados en puntaje. Además, se realizaron *benchmarks* experimentales utilizando distintos tamaños de entrada, con el objetivo de comparar el comportamiento y el rendimiento de los algoritmos implementados en escenarios de mejor caso, peor caso y caso promedio.

OBJETIVOS

2.1. Objetivo Principal

Ampliar el sistema de gestión de deportistas desarrollado en el proyecto anterior, incorporando algoritmos de ordenamiento, búsqueda y selección basados en la estrategia de *Divide y Vencerás*, con el propósito de analizar su comportamiento teórico y experimental sobre distintos tamaños de entrada y aplicarlos a funcionalidades concretas del sistema, tales como ordenamiento por criterios, búsqueda de registros, generación de rankings, obtención del k -ésimo mejor deportista y consulta de deportistas dentro de un rango de puntaje.

2.2. Objetivos Secundarios

- Implementar algoritmos de ordenamiento basados en *Divide y Vencerás*, específicamente *Merge Sort*, *Merge-Insertion Sort* y *Quick Sort*, permitiendo ordenar deportistas según distintos criterios como ID, puntaje, competencias, nombre y equipo.
- Implementar distintas variantes de *Quick Sort* utilizando partición de Lomuto, considerando la selección del pivote como último elemento, primer elemento, elemento aleatorio y mediana de tres, con el fin de evaluar cómo esta decisión afecta el rendimiento del algoritmo.
- Implementar algoritmos de búsqueda sobre arreglos ordenados, incluyendo búsqueda binaria recursiva, búsqueda binaria por rangos, búsqueda exponencial y búsqueda por interpolación, aplicándolos sobre los datos del sistema de deportistas.
- Implementar el algoritmo *Quick Select* para obtener el k -ésimo mejor deportista según un criterio determinado, especialmente el puntaje, evitando ordenar completamente el arreglo cuando solo se requiere una posición específica.
- Integrar los algoritmos desarrollados al sistema existente, manteniendo la generación, carga y almacenamiento de registros en archivos CSV, así como las opciones de menú y ejecución por línea de comandos.
- Diseñar y ejecutar *benchmarks* experimentales que permitan medir los tiempos de ejecución de los algoritmos implementados en distintos tamaños de entrada y en escenarios de mejor caso, caso promedio y peor caso.
- Comparar los resultados experimentales obtenidos con el análisis teórico de complejidad temporal, identificando coincidencias, diferencias prácticas, efectos de las variantes implementadas y posibles li-

mitaciones de la medición.

MARCO TEÓRICO

Los algoritmos de ordenamiento, búsqueda y selección cumplen un rol fundamental en el procesamiento de datos, ya que permiten organizar información, localizar registros específicos y obtener elementos relevantes dentro de un conjunto. En el proyecto anterior se trabajó principalmente con algoritmos iterativos clásicos, tales como Bubble Sort optimizado, Insertion Sort, Selection Sort optimizado, Cocktail Shaker Sort, búsqueda secuencial y búsqueda binaria iterativa. Estos algoritmos permitieron construir una primera base funcional para manipular registros de deportistas y analizar su comportamiento teórico y experimental.

El presente proyecto corresponde a una ampliación de dicho sistema, incorporando algoritmos basados en la estrategia de Divide y Vencerás. Esta técnica consiste en dividir un problema en subproblemas más pequeños, resolverlos de forma recursiva y, cuando corresponde, combinar sus resultados para obtener la solución final. Su importancia radica en que permite diseñar algoritmos más eficientes que muchas soluciones iterativas directas, especialmente cuando el tamaño de entrada crece considerablemente.

3.1. Algoritmos de Ordenamiento

Dentro de los algoritmos de ordenamiento incorporados en esta etapa se encuentra Merge Sort. Este algoritmo divide el arreglo en dos mitades, ordena recursivamente cada una de ellas y luego combina ambos segmentos mediante un proceso de mezcla. Su complejidad temporal es $O(n \log n)$ en el mejor, promedio y peor caso, debido a que el arreglo se divide logarítmicamente y en cada nivel se realiza una combinación lineal de los elementos. Sin embargo, requiere memoria auxiliar para realizar la mezcla, lo que representa una diferencia importante frente a otros algoritmos de ordenamiento que trabajan principalmente sobre el mismo arreglo. Además de la versión clásica de Merge Sort, se considera una versión optimizada que utiliza Insertion Sort cuando los subarreglos alcanzan un tamaño pequeño definido por un umbral. Esta optimización se justifica porque Insertion Sort presenta un buen rendimiento práctico en arreglos pequeños o casi ordenados, aun cuando su peor caso teórico sea $O(n^2)$.

Otro algoritmo de ordenamiento relevante es Quick Sort. Su funcionamiento se basa en seleccionar un pivote, particionar el arreglo en torno a dicho elemento y ordenar recursivamente las secciones resultantes. En el caso promedio, Quick Sort presenta complejidad $O(n \log n)$, pero en el peor caso puede degradarse a $O(n^2)$ si las particiones resultan demasiado desbalanceadas. La elección del pivote es un factor determinante en el rendimiento de Quick Sort. Por esta razón, se consideran distintas variantes: pivote como último elemento, primer elemento, elemento aleatorio y mediana de tres. El pivote fijo, como el primero o el último elemento, puede generar malos resultados cuando los datos ya se encuentran ordenados o invertidos. En cambio, el pivote aleatorio reduce la probabilidad de caer repetidamente en particiones desfavorables. La mediana de tres busca seleccionar un pivote más representativo a partir del primer, medio y último elemento del segmento, con el objetivo de mejorar el balance de las particiones.

3.2. Algoritmos de Búsqueda

Este proyecto amplía la búsqueda binaria trabajada anteriormente mediante una versión recursiva. La búsqueda binaria divide el espacio de búsqueda en mitades sucesivas, comparando el valor objetivo con el elemento central del arreglo. Su complejidad temporal es $O(\log n)$ en el peor caso, siempre que los datos se encuentren previamente ordenados. La versión recursiva mantiene la misma complejidad que la iterativa, pero expresa de forma más directa la estrategia de Divide y Vencerás.

También se incorpora la búsqueda binaria para rangos, cuyo objetivo no es encontrar únicamente una coincidencia exacta, sino determinar los límites de un conjunto de valores dentro de un arreglo ordenado. En el contexto del sistema de deportistas, esta variante permite localizar aquellos registros cuyo puntaje se encuentra dentro de un intervalo definido. Para ello, se pueden aplicar dos búsquedas binarias modificadas: una para encontrar la primera posición válida dentro del rango y otra para encontrar la última posición válida. Esto permite obtener los límites del segmento en tiempo $O(\log n)$, sin recorrer inicialmente todo el arreglo.

La búsqueda exponencial corresponde a una estrategia híbrida que combina crecimiento exponencial con búsqueda binaria. Primero determina un rango probable donde podría encontrarse el elemento buscado, aumentando el límite superior de búsqueda en potencias de dos. Una vez identificado el intervalo, aplica búsqueda binaria dentro de ese segmento. Este algoritmo resulta útil cuando el elemento buscado se encuentra cerca del inicio del arreglo, ya que puede reducir rápidamente el espacio de búsqueda. Su complejidad en el peor caso es $O(\log n)$ sobre arreglos ordenados.

Por otro lado, la búsqueda por interpolación estima la posición probable del elemento buscado mediante una fórmula basada en la relación entre el valor objetivo y los valores ubicados en los extremos del rango actual. A diferencia de la búsqueda binaria, que siempre revisa el punto medio, la búsqueda por interpolación intenta aproximarse a la ubicación real del dato. Su rendimiento puede ser muy eficiente cuando los valores están distribuidos de manera uniforme, llegando a un caso promedio cercano a $O(\log \log n)$. Sin embargo, si la distribución de los datos no es uniforme, su rendimiento puede deteriorarse hasta $O(n)$ en el peor caso.

3.3. Algoritmos de Selección

Finalmente, se incorpora Quick Select como algoritmo de selección. Este algoritmo permite encontrar el k -ésimo elemento de un arreglo según un criterio determinado, sin necesidad de ordenar completamente todos los datos. Su lógica se basa en la misma partición utilizada por Quick Sort: después de ubicar el pivote en su posición final, el algoritmo continúa únicamente por el subarreglo donde se encuentra la posición buscada. En promedio, Quick Select tiene complejidad $O(n)$, ya que descarta una parte del arreglo en cada partición. No obstante, en el peor caso puede alcanzar $O(n^2)$ si las particiones son muy desbalanceadas. En el contexto del sistema desarrollado, Quick Select resulta especialmente útil para mostrar el k -ésimo mejor deportista según puntaje (realizar un ranking), ya que evita ordenar completamente el arreglo cuando solo se requiere un elemento específico.

El análisis experimental de estos algoritmos permite contrastar su complejidad teórica con su rendimiento

real. Para ello, se utilizan benchmarks sobre distintos tamaños de entrada y diferentes escenarios, tales como mejor caso, caso promedio y peor caso. Los tiempos obtenidos se almacenan en archivos CSV y posteriormente se representan mediante gráficos, lo que facilita comparar el comportamiento de cada algoritmo. De esta manera, el proyecto no solo busca implementar correctamente las técnicas de Divide y Vencerás, sino también interpretar sus ventajas, limitaciones y diferencias frente a los algoritmos iterativos desarrollados en la etapa anterior.

DESARROLLO

4.1. Diseño general del sistema

El sistema fue diseñado para gestionar información de deportistas de forma simple y modular, permitiendo generar datos, cargarlos desde archivos CSV y aplicar sobre ellos distintas operaciones de ordenamiento, búsqueda y análisis experimental. Para facilitar su desarrollo y comprensión, la implementación se organizó en componentes que separan la representación de los datos, la generación y carga de registros, y las funcionalidades principales ofrecidas por el programa.

4.1.1. Estructura de datos de los deportistas

La estructura de datos utilizada para representar a cada deportista se definió como una estructura con campos específicos para almacenar distinta información; estos campos corresponden a:

- **ID:** Identificador único del deportista.
- **Nombre:** Nombre completo del deportista.
- **Equipo:** Equipo al que pertenece.
- **Puntaje:** Puntuación total acumulada.
- **Competencias:** Número de competencias en las que ha participado.

Cada parámetro permite almacenar información, la cual puede ser utilizada para ordenar o buscar a los deportistas según distintos criterios. Esta estructura se implementó utilizando un **struct** en C, lo que facilita su manejo y acceso a los campos correspondientes durante las operaciones de ordenamiento y búsqueda.

4.1.2. Generación y carga de datos

La generación de datos de los parámetros asociados a deportistas se realizó mediante distintas funciones, las cuales permiten crear u obtener estos registros de manera pseudoaleatoria. La generación de un nombre se realizó permitiendo seleccionar aleatoriamente letras de un conjunto predefinido, escogiendo un tamaño de 3 a 8 caracteres. Para el equipo, se seleccionó aleatoriamente entre un conjunto específico de equipos de Esports. El ID se generó en forma incremental, asegurando la unicidad de cada registro. Por otro lado, el puntaje y la cantidad de competencias se generaron utilizando funciones de generación de números aleatorios dentro de rangos predefinidos. Esta metodología permitió crear un conjunto de datos variado y representativo para probar las funcionalidades del sistema. Posterior a la creación de los registros, estos se desordenaron utilizando el algoritmo de Fisher-Yates para garantizar que los datos no se encuentren en un orden específico, lo que es fundamental para evaluar correctamente el desempeño de los algoritmos de ordenamiento y búsqueda implementados. Finalmente, los registros generados se almacenaron en un archivo CSV, lo que facilita su posterior carga y manipulación dentro del programa.

4.1.3. Funcionalidades del sistema

Una vez cargados los datos de los deportistas, el programa permite realizar distintas operaciones combinando directrices de línea de comandos con menús interactivos. Entre las funcionalidades principales se encuentra el ordenamiento de los datos de los deportistas de manera ascendente o descendente utilizando el ID, puntaje, cantidad de competencias, nombre y equipo. Para ello, se preselecciona el algoritmo de búsqueda mas optimo. Otra funcionalidad importante es la búsqueda de deportistas por ID, la cual puede realizarse mediante búsqueda binaria recursiva, búsqueda exponencial o búsqueda por interpolación. Para esta operación, el programa selecciona el algoritmo de búsqueda más adecuado. Además, el sistema permite mostrar un ranking con los mejores N deportistas de acuerdo a su puntaje, lo que logra una aplicación práctica del ordenamiento de datos dentro del contexto del problema planteado. En adición a esto, se implemento la opción de mostrar deportistas dentro de un rango de puntajes específicos. Se utilizo la búsqueda binaria por rangos para esto. Junto con las operaciones anteriores, el programa incorpora la ejecución de benchmarks para medir el tiempo de ejecución de los algoritmos de ordenamiento y búsqueda. Estos resultados se almacenan en archivos CSV y posteriormente pueden utilizarse para generar gráficos comparativos mediante gnuplot, facilitando el análisis experimental de los algoritmos implementados. En conjunto, estas funcionalidades permiten que el sistema no solo cumpla con los requerimientos prácticos de manipulación de datos, sino que también sirva como base para el estudio comparativo del comportamiento de los algoritmos.

4.2. Algoritmos de ordenamiento

Existen numerosos algoritmos de ordenamiento, desde algunos muy eficientes, capaces de alcanzar una complejidad temporal de apenas $O(n)$, como *Radix Sort*, hasta otros extremadamente ineficientes, que incluso pueden llegar a $O(n \times n!)$, como es el caso de *Bogo Sort*. Sin embargo, en este trabajo nos centramos únicamente en *Merge Sort* y *Quick Sort*, dos algoritmos relativamente eficientes que emplean el principio de "Divide y Vencerás".

4.2.1. Merge Sort

Merge Sort, también conocido como 'Ordenamiento por Mezcla', es un algoritmo de ordenamiento recursivo, que en cada paso va dividiendo el arreglo en 2 subarreglos de aproximadamente la mitad del tamaño anterior, y eso lo hace hasta que todos los subarreglos sean de tamaño 1. Cuando eso sucede, se comienzan a combinar pares de subarreglos adyacentes, recorriéndolos linealmente para fusionarlos en el orden correcto, hasta que finalmente todo el arreglo queda ordenado.

En base al principio en el que se fundamenta *Merge Sort*, es posible deducir su complejidad temporal mediante la siguiente ecuación de recurrencia:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Esto se debe a que el problema se divide recursivamente en 2 subproblemas de tamaño $n/2$, mientras que el costo adicional corresponde al proceso de fusión de los subarreglos, el cual requiere recorrer linealmente

los elementos para ubicarlos en el orden correcto, por lo que dicho costo es de complejidad lineal.

Para resolver la ecuación de recurrencia, se puede emplear el método de iteración. Considerando una constante k para representar el costo lineal, se tiene:

$$T(n) = 2T\left(\frac{n}{2}\right) + kn$$

A partir de esto, la expresión equivalente para $T\left(\frac{n}{2}\right)$ sería:

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{4}\right) + k\left(\frac{n}{2}\right)$$

Entonces, reemplazando en la ecuación original, se obtiene:

$$T(n) = 2\left(2T\left(\frac{n}{4}\right) + k\left(\frac{n}{2}\right)\right) + kn$$

Simplificando:

$$T(n) = 4T\left(\frac{n}{4}\right) + 2kn$$

Ahora, realizando otra iteración, se tiene que:

$$T\left(\frac{n}{4}\right) = 2T\left(\frac{n}{8}\right) + k\left(\frac{n}{4}\right)$$

Reemplazando nuevamente:

$$T(n) = 4\left(2T\left(\frac{n}{8}\right) + k\left(\frac{n}{4}\right)\right) + 2kn$$

$$T(n) = 8T\left(\frac{n}{8}\right) + 3kn$$

Continuando de esta forma, se puede generalizar que:

$$T(n) = 2T\left(\frac{n}{2}\right) + kn = 4T\left(\frac{n}{4}\right) + 2kn = 8T\left(\frac{n}{8}\right) + 3kn = 2^i T\left(\frac{n}{2^i}\right) + ikn$$

La recursión finaliza cuando se alcanza el caso base, es decir, cuando los subarreglos tienen tamaño 1. Por lo tanto, debe cumplirse que:

$$\frac{n}{2^i} = 1$$

Despejando:

$$i = \log n$$

Reemplazando esto en la expresión general obtenida anteriormente:

$$T(n) = 2^{\log n} T(1) + \log n kn$$

Como $2^{\log n} = n$, entonces:

$$T(n) = nT(1) + kn \log n$$

Finalmente, considerando que $T(1)$ y k son constantes, se concluye que:

$$T(n) \in O(n \log n)$$

Por lo tanto, *Merge Sort* posee complejidad temporal $O(n \log n)$ tanto en el mejor caso, como en el caso promedio y peor caso, ya que el arreglo siempre se divide de manera uniforme, independientemente del orden inicial de los elementos.

Aun así, es posible distinguir ciertas diferencias en la cantidad de comparaciones realizadas durante la etapa de fusión (*Merge*). El mejor caso ocurre cuando los elementos de uno de los subarreglos se consumen rápidamente, como sucede cuando el arreglo ya se encuentra previamente ordenado, reduciendo así la cantidad de comparaciones necesarias. En cambio, el peor caso ocurre cuando los elementos de ambos subarreglos se encuentran altamente intercalados, obligando al algoritmo a realizar prácticamente el máximo número de comparaciones posibles durante la fusión. El caso promedio corresponde a una situación intermedia entre ambos escenarios.

4.2.1.1. Implementacion Merge Sort

En este proyecto, la implementación de *Merge Sort* permite la búsqueda de deportistas por distintos parámetros, como el ID, el puntaje, la cantidad de competencias, el nombre o el equipo. Para esto, se implementó una función de comparación que recibe un parámetro adicional que indica el criterio de ordenamiento a utilizar. De esta manera, se puede reutilizar la misma implementación de *Merge Sort* para ordenar por distintos parámetros, simplemente cambiando la lógica de comparación según el criterio seleccionado. Además, se implementó la opción de ordenar tanto en forma ascendente como descendente, lo que se logra modificando la lógica de comparación para invertir el orden cuando se desea ordenar de manera descendente.

4.2.2. Merge-Insertion Sort

Merge-Insertion Sort corresponde a una variante optimizada de *Merge Sort*, cuyo objetivo es reducir el *overhead* recursivo que se produce al trabajar con subarreglos muy pequeños.

La idea principal consiste en que, cuando los subarreglos alcanzan un tamaño suficientemente pequeño, en lugar de continuar dividiéndolos recursivamente, se utiliza *Insertion Sort* para ordenarlos directamente.

Esto se hace debido a que *Insertion Sort*, a pesar de poseer complejidad $O(n^2)$ en el peor caso, tiene un rendimiento muy eficiente para arreglos de tamaño reducido, ya que posee un *overhead* muy bajo y además aprovecha favorablemente arreglos parcialmente ordenados.

De esta manera, *Merge-Insertion Sort* busca combinar las ventajas de ambos algoritmos: la eficiencia asintótica de *Merge Sort* para arreglos grandes, junto con la simplicidad y rapidez de *Insertion Sort* en subarreglos pequeños.

En términos de complejidad asintótica, esta variante sigue perteneciendo a $O(n \log n)$, aunque en la práctica puede presentar mejoras de rendimiento respecto a la versión clásica de *Merge Sort*, dependiendo del umbral utilizado para decidir cuándo aplicar *Insertion Sort*, que es uno de los aspectos que se analizará en el presente trabajo.

4.2.2.1. Implementacion Merge-Insertion Sort

Al igual que con la implementacion de *Merge Sort*, la implementacion de merge-insertion sort permite ordenar por distintos parametros, utilizando la misma funcion de comparacion y eleccion del sentido del ordenamiento. Ademas de esto se permite la entrega por paso de parametro del treshhold o criterio para utilizar insertion sort en sub-arreglos.

4.2.3. Quick Sort

Quick Sort es un algoritmo que consiste en seleccionar un pivote dentro del arreglo, y mediante comparaciones e intercambios al recorrer el arreglo, ir colocando los elementos que sean menores al pivote, a su izquierda, y los que sean mayores, a su derecha. Y luego con las particiones (subarreglos) que hayan quedado, volver a hacer lo mismo, hasta llegar al caso trivial que es cuando las particiones llegan a tener a lo más un elemento.

Existen varias formas de seleccionar el pivote en este algoritmo. Estas son:

- El primer elemento.
- El último elemento.
- Un elemento aleatorio.
- Mediante mediana de tres.

El mejor caso ocurre cuando el límite que separan las 2 particiones siempre ocurre prácticamente por la mitad. En ese caso, la complejidad temporal del algoritmo viene dada por la recurrencia:

$$T(n) = 2T(\frac{n}{2}) + O(n)$$

Esta recurrencia, es la misma que se obtuvo para *Merge Sort*, por lo cual el mejor caso de *Quick Sort* es el mismo que para *Merge Sort*, o sea $O(n \log(n))$.

El caso promedio ocurre cuando las particiones de *Quick Sort*, quedan un poquito desequilibradas pero no demasiado. Un caso promedio aproximado podría ser:

$$T(n) = T(\frac{n}{3}) + T(\frac{2n}{3}) + O(n)$$

Esta recurrencia también tiene una solución $T(n) \in O(n \log n)$, por lo que la complejidad temporal para *Quick Sort* en caso promedio también es $O(n \log(n))$.

El peor caso ocurre por ejemplo cuando siempre la partición queda extremadamente desequilibrada, con 1 elemento en uno de los lados, y los otros $n-1$ del otro lado, por lo que la recurrencia para ese caso sería:

$$T(n) = T(n - 1) + O(n)$$

Resolviendo esa recurrencia, se llega a que en el peor caso, *Quick Sort* tiene una complejidad temporal de $O(n^2)$.

Un ejemplo de peor caso, es cuando el arreglo ya está ordenado y el método de seleccionar el pivote, es el primer elemento, o el último.

4.2.3.1. Implementacion Quick Sort

Para la implementacion de Quick Sort se decidio utilizar el metodo de seleccion de pivote de lomuto, el cual selecciona el ultimo elemento del arreglo como pivote. Esto eleccion se realizo teniendo en cuenta la facilidad de implementacion y la eficiencia que presenta en arreglos desordenados. Al igual que los algoritmos de ordenamiento anteriores, se implemento la opcion de ordenar por distintos parametros, utilizando la misma funcion de comparacion y eleccion del sentido del ordenamiento.

4.3. Algoritmos de búsqueda

4.3.1. Búsqueda binaria recursiva

La búsqueda binaria recursiva es un algoritmo de búsqueda, que al igual que los otros que se implementaron, utiliza el paradigma de "**Divide y Vencerás**". Funciona solo si el arreglo ya fue ordenado previamente. La lógica del algoritmo consiste en comparar el elemento buscado con el elemento ubicado en la posición central del arreglo. Si ambos elementos coinciden, la búsqueda finaliza exitosamente. En caso contrario,

si el elemento central es mayor que el valor buscado, se descarta la mitad superior del arreglo y la búsqueda continúa únicamente sobre la primera mitad. Por otro lado, si el elemento central es menor que el valor buscado, se descarta la mitad inferior y la búsqueda prosigue sobre la segunda mitad del arreglo. Este proceso se repite recursivamente hasta encontrar el elemento o hasta que el rango de búsqueda se vuelva vacío.

El peor caso para la búsqueda binaria ocurre cuando el elemento buscado no se encuentra en el arreglo, o cuando se encuentra en una de las últimas divisiones realizadas por el algoritmo. En este escenario, el algoritmo debe continuar reduciendo el espacio de búsqueda hasta llegar a un subarreglo vacío o de tamaño 1.

El caso promedio ocurre cuando el elemento buscado se encuentra luego de una cantidad intermedia de divisiones del arreglo. Debido a que el algoritmo reduce el espacio de búsqueda aproximadamente a la mitad en cada paso, el número de comparaciones sigue creciendo de forma logarítmica respecto al tamaño del arreglo.

Por lo tanto, tanto el caso promedio como el peor caso presentan una complejidad temporal de $O(\log n)$

4.3.1.1. Implementacion Busqueda Binaria

Para la implementacion de la busqueda binaria recursiva, se decidio utilizar una funcion que requiere el arreglo de deportistas, el elemento a buscar (en el caso de los algoritmos de busqueda, este corresponde a un ID), el indice de inicio y el indice de fin del rango de busqueda. La funcion se llama a si misma recursivamente, ajustando los indices de inicio y fin segun corresponda, hasta encontrar el elemento buscado o hasta que el rango de busqueda se vuelva invalido.

4.3.2. Búsqueda exponencial

La búsqueda exponencial es un algoritmo de búsqueda que combina una fase de expansión exponencial con búsqueda binaria. Al igual que la búsqueda binaria, requiere que los datos se encuentren previamente ordenados.

La lógica del algoritmo consiste en revisar secuencialmente elementos ubicados en posiciones que crecen exponencialmente, es decir, en posiciones de la forma:

$$k = 2^i, \quad \forall i \in \mathbb{N}$$

mientras el valor revisado sea menor que el elemento buscado y el índice permanezca dentro de los límites del arreglo.

Si el algoritmo encuentra directamente el elemento buscado en alguna de estas posiciones, la búsqueda finaliza inmediatamente. En caso contrario, cuando encuentra un elemento mayor que el valor buscado, se determina un rango acotado entre la última potencia de 2 revisada y la actual, aplicando posteriormente búsqueda binaria sobre dicho intervalo.

También puede ocurrir que el elemento buscado sea mayor que todos los elementos del arreglo. En ese caso, la fase exponencial continúa aumentando el índice hasta alcanzar o sobrepasar el tamaño del arreglo. Cuando esto sucede, el límite superior de búsqueda se ajusta al último índice válido del arreglo antes de

aplicar búsqueda binaria sobre el intervalo restante.

El peor caso ocurre cuando el elemento buscado se encuentra cerca del final del arreglo, o sea cuando es mayor que la mayoría de los elementos almacenados. En ese caso, el algoritmo debe recorrer sucesivamente todas las posiciones que correspondan a potencias de 2 dentro de los límites del arreglo antes de determinar el rango final de búsqueda y aplicar búsqueda binaria sobre dicho intervalo.

Debido a que la fase de expansión exponencial realiza a lo más $O(\log n)$ comparaciones, y posteriormente la búsqueda binaria también posee complejidad $O(\log n)$, la complejidad temporal total del algoritmo en el peor caso es $O(\log n)$

El caso promedio ocurre cuando el elemento buscado se encuentra en una posición intermedia del arreglo, de manera que la fase exponencial logra acotar relativamente rápido el intervalo donde podría encontrarse el dato. Posteriormente, la búsqueda binaria se aplica únicamente sobre dicho rango acotado. Al igual que en el peor caso, la complejidad temporal promedio de la búsqueda exponencial es $O(\log n)$.

4.3.2.1. Implementacion Búsqueda Exponencial

En el caso de la implementación de la búsqueda exponencial, se decidió utilizar una función que recibe el arreglo de deportistas, el elemento a buscar (ID), y el tamaño del arreglo. La función comienza revisando el primer elemento del arreglo, y luego va incrementando el índice de manera exponencial, hasta encontrar un elemento mayor que el buscado o hasta que el índice sobrepase el tamaño del arreglo. Luego, se determina el rango acotado para aplicar búsqueda binaria, y se llama a la función de búsqueda binaria recursiva para realizar la búsqueda dentro de dicho rango.

4.3.3. Búsqueda por interpolación

La búsqueda por interpolación es un algoritmo de búsqueda que requiere que los datos se encuentren previamente ordenados. Su funcionamiento consiste en estimar la posición probable del elemento buscado, suponiendo que los datos se encuentran distribuidos de manera uniforme.

La posición estimada se calcula mediante la siguiente fórmula:

$$supposed_idx = start_idx + \frac{element - array[start_idx]}{array[end_idx] - array[start_idx]} \times (end_idx - start_idx)$$

A diferencia de la búsqueda binaria, que siempre revisa el elemento central del arreglo, la búsqueda por interpolación intenta aproximarse directamente a la posición donde probablemente se encuentre el dato buscado. Esto puede mejorar considerablemente el rendimiento cuando los datos están distribuidos uniformemente.

El peor caso para este algoritmo ocurre cuando los datos presentan una distribución muy desigual o altamente sesgada hacia alguno de los extremos del arreglo. Por ejemplo al tener algo así:

```
int array [] = {1, 2, 3, 4, 5, 1000, 10000};
```

Si se desea buscar el elemento 1000, la fórmula de interpolación estimará inicialmente una posición muy cercana al inicio del arreglo, debido a la gran diferencia entre los últimos elementos y el resto de los datos. Como consecuencia, las estimaciones realizadas serán poco precisas y el algoritmo reducirá muy lentamente el espacio de búsqueda, pudiendo degradarse hasta una complejidad temporal de $O(n)$.

El caso promedio ocurre cuando los datos están parcialmente uniformes y sin valores que se escapen demasiado de los demás. En ese caso la complejidad temporal es de apenas $O(\log(\log n))$.

4.3.3.1. Implementacion Búsqueda por Interpolacion

Para la implementacion de la búsqueda por interpolacion, se decidió utilizar una funcion que reciba el arreglo de deportistas, el indice de inicial y final del rango de búsqueda y el elemento a buscar (ID). La funcion calcula la posicion estimada utilizando la formula de interpolacion, y luego compara el elemento ubicado en dicha posicion con el valor buscado. Si ambos coinciden, la búsqueda finaliza exitosamente. En caso contrario, si el elemento en la posicion estimada es mayor que el valor buscado, se ajusta el indice final del rango de búsqueda a la posicion estimada menos uno. Por otro lado, si el elemento en la posicion estimada es menor que el valor buscado, se ajusta el indice inicial del rango de búsqueda a la posicion estimada mas uno. Luego, se llama recursivamente a la funcion de búsqueda por interpolacion con los indices ajustados, hasta encontrar el elemento o hasta que el rango de búsqueda se vuelva invalido. Es importante destacar que para utilizar este algoritmo, es necesario que el arreglo se encuentre previamente ordenado, ya que la formula de interpolacion se basa en la suposicion de que los datos estan distribuidos de manera uniforme. Si el arreglo no esta ordenado, la formula de interpolacion no sera precisa y el algoritmo no funcionara correctamente.

4.3.4. Búsqueda Binaria por Rangos

La búsqueda binaria por rangos es una extensión natural de la búsqueda binaria que, en lugar de buscar un elemento específico, busca todos los elementos dentro de un rango de valores definido por un límite inferior y un límite superior. Esta operación es especialmente útil en contextos donde se necesita recuperar múltiples registros que caen dentro de ciertos parámetros, como por ejemplo encontrar todos los deportistas cuyo puntaje esté entre 100 y 500 puntos.

El algoritmo funciona en dos fases principales. En la primera fase, se determina el índice del primer elemento cuyo valor es mayor o igual al límite inferior del rango. En la segunda fase, se determina el índice del último elemento cuyo valor es menor o igual al límite superior del rango. Una vez que se conocen ambos índices, todos los elementos entre ellos (inclusive) forman el resultado deseado.

Para implementar cada una de estas fases se utilizan variantes especializadas de búsqueda binaria que, en lugar de buscar un valor exacto, buscan una posición límite. Específicamente, en la primera fase se ejecuta una búsqueda que mantiene el mejor candidato encontrado hasta el momento mientras se divide el espacio de búsqueda, asegurando que cuando termine la recursión, se tendrá el índice del primer elemento que cumple la condición. De manera análoga, en la segunda fase se ejecuta una búsqueda simétrica buscando desde el lado contrario.

En términos de complejidad temporal, dado que se ejecutan dos búsquedas binarias independientes, la complejidad total es de $O(\log n)$ para buscar ambos límites, más $O(k)$ para procesar los resultados, donde k es la cantidad de elementos dentro del rango. En el peor caso, si el rango abarca prácticamente todos los elementos del arreglo, k puede ser igual a n , resultando en una complejidad total de $O(n)$. Sin embargo, en casos promedio donde el rango es relativamente pequeño, la complejidad se aproxima a $O(\log n)$.

4.3.4.1. Implementacion Búsqueda Binaria por Rangos

La implementacion de la busqueda binaria por rangos dentro del proyecto, consiste en utilizar dos funciones especializadas, una para encontrar el índice del primer elemento que cumple la condición de ser mayor o igual al límite inferior, y otra para encontrar el índice del último elemento que cumple la condición de ser menor o igual al límite superior. Ambas funciones utilizan una lógica similar a la búsqueda binaria tradicional, pero con ajustes para mantener un candidato válido durante la división del espacio de búsqueda. Luego de obtener ambos índices, se puede iterar sobre el rango definido por ellos para procesar o mostrar los elementos que cumplen con los criterios establecidos.

4.4. Algoritmos de selección

4.4.1. Quick Select

Quick Select es un algoritmo de selección basado en el paradigma de "**Divide y Vencerás**", cuyo objetivo consiste en encontrar el elemento que ocuparía la posición k -ésima dentro de un arreglo ordenado, sin necesidad de ordenar completamente todos los elementos.

El algoritmo utiliza una estrategia similar a *Quick Sort*, seleccionando un pivote y reorganizando el arreglo mediante una partición. Luego de realizar la partición, el pivote queda ubicado en su posición definitiva dentro del arreglo ordenado.

Si la posición del pivote coincide con la posición k buscada, el algoritmo finaliza inmediatamente. En caso contrario, si k es menor que la posición del pivote, el algoritmo continúa recursivamente sobre la partición izquierda. Por otro lado, si k es mayor, la búsqueda continúa únicamente sobre la partición derecha.

A diferencia de *Quick Sort*, *Quick Select* no necesita procesar ambas particiones recursivamente, sino solamente aquella donde podría encontrarse el elemento buscado. Debido a esto, el algoritmo suele presentar un mejor rendimiento práctico para problemas de selección.

El peor caso ocurre cuando el pivote genera particiones extremadamente desbalanceadas, dejando un subarreglo de tamaño $n - 1$ y otro de tamaño 0. Esto puede suceder, por ejemplo, al utilizar el primer o último elemento como pivote sobre arreglos previamente ordenados. En este caso, la recurrencia queda dada por:

$$T(n) = T(n - 1) + O(n), \text{ cuya solución corresponde a una complejidad temporal de } O(n^2).$$

El caso promedio ocurre cuando las particiones quedan relativamente balanceadas, descartando aproximadamente la mitad de los elementos en cada iteración. La recurrencia aproximada para este caso es:

$$T(n) = T\left(\frac{n}{2}\right) + O(n), \text{ cuya solución corresponde a } O(n)$$

4.4.1.1. Implementacion Quick Select

Para la implementacion de Quick Select, se decidio permitir la eleccion del tipo de pivote, estando disponible el primer elemento, el ultimo elemento, un elemento aleatorio o la mediana de tres. Esta decision se tomo para poder comparar el comportamiento del algoritmo bajo distintos escenarios experimentales y reutilizar una misma estructura de particion. Al igual que con los algoritmos de ordenamiento, se implemento la opcion de seleccionar por distintos parametros, utilizando una funcion de comparacion adaptada al criterio elegido.

Ademas de esto, la implementacion recibe el indice inicial y el indice final del rango de busqueda, de forma similar a una busqueda recursiva sobre subarreglos. Luego de particionar el arreglo, la funcion compara la posicion del pivote con la posicion buscada; si coincide, se devuelve el elemento, y si no, se continua recursivamente solo sobre la mitad donde puede encontrarse el resultado. De esta manera, Quick Select evita ordenar completamente el arreglo y mantiene el enfoque de dividir el problema en subproblemas mas pequeños hasta encontrar el elemento requerido.

4.5. Benchmarks, Generacion de resultados y graficos

Los benchmarks incluidos (búsqueda, ordenamiento, selección, búsqueda por rango y evaluación de thresholds para merge+insertion) miden tiempos en función de n usando intervalos uniformes y los casos mejor/promedio/peor; los resultados se promedian sobre varias repeticiones y se exportan a CSV para su posterior graficado. Para obtener medidas fiables en operaciones muy rápidas se emplearon bucles de muchas iteraciones por llamada y una variable tipo `volatile` para preservar los resultados frente a optimizaciones, además de repetir cada experimento y promediar los tiempos para reducir el ruido.

Para la evaluación del rendimiento de cada algoritmo, se implementaron benchmarks para ordenamiento, búsqueda, búsqueda por rangos, quick select y merge insertion con distintos thresholds. Para cada uno de estos benchmarks, se midió el tiempo de ejecución utilizando la función `clock()` de la biblioteca `time.h`, y se almacenaron los resultados en archivos CSV. Posteriormente, estos archivos fueron utilizados para generar graficos comparativos mediante gnuplot, lo que permitió visualizar claramente las diferencias de rendimiento entre los algoritmos implementados bajo distintas condiciones y tamaños de entrada.

En algunos casos, como en el benchmark de búsqueda para el caso de la búsqueda por interpolación, el peor caso fue generado modificando a la fuerza el elemento anterior al elemento buscado, esto debido a que la fórmula de interpolación se basa en la suposición de que los datos están distribuidos de manera uniforme, por lo que al modificar el elemento anterior al buscado, se logra que la fórmula de interpolación no sea precisa y el algoritmo tenga un rendimiento muy bajo, lo que permite medir su comportamiento en el peor caso. En otros casos, como en el benchmark de búsqueda por rango, el caso promedio fue generado seleccionando un elemento aleatorio dentro del rango de puntajes, esto debido a que al seleccionar un elemento aleatorio, se logra obtener una muestra representativa del comportamiento del algoritmo en condiciones promedio, ya que se pueden obtener resultados variados dependiendo del elemento seleccionado y su posición dentro del arreglo. De esta manera, se busca obtener una evaluación más completa y realista del rendimiento de los algoritmos implementados.

En el caso del benchmark de merge-insertion, se evaluaron distintos thresholds para determinar el punto óptimo en el cual aplicar insertion sort en lugar de continuar con la división recursiva. Para esto, se midió el tiempo de ejecución del algoritmo utilizando diferentes valores de threshold, y se compararon los resultados para identificar cuál de ellos ofrece el mejor rendimiento en función del tamaño de entrada.

4.5.1. Generacion de graficos

Para generar los graficos, se utilizó gnuplot, cargando los datos de los benchmark previamente almacenados en CSV. Creando así graficos comparativos que permiten visualizar la evolución del tiempo de ejecución de cada algoritmo en función del tamaño de entrada, y comparando su comportamiento en los casos mejor, promedio y peor. Además, se generaron graficos con distintas escalas (lineal y logarítmica) para facilitar la visualización de las diferencias de rendimiento entre los algoritmos, especialmente en casos donde las diferencias pueden ser significativas. De esta manera, se busca obtener una representación visual clara y efectiva del comportamiento de cada algoritmo bajo distintas condiciones experimentales.

4.6. Análisis de Resultados

A continuación, se mostrarán los distintos gráficos obtenidos luego de ejecutar los programas y se analizará el rendimiento y la complejidad algorítmica de cada uno de los algoritmos implementados.

4.6.1. Comparación de algoritmos de ordenamiento

4.6.1.1. Análisis de mejor caso

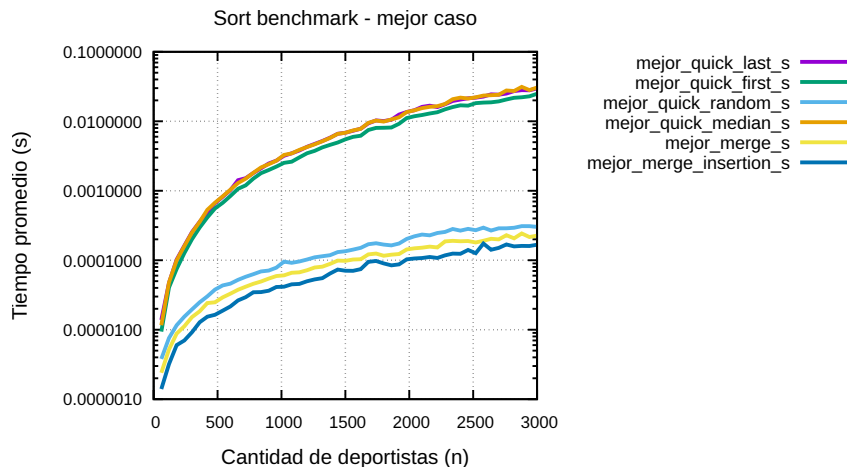


Figura 4.1: Benchmark de algoritmos de ordenamiento en mejor caso.

En este primer gráfico, se visualiza el comportamiento de los algoritmos de ordenamiento implementados en el mejor caso. Cabe destacar que en la implementación realizada, el **mejor caso** implementado en realidad fue uno genérico para los algoritmos de ordenamiento, y se definió que ese mejor caso sería cuando el arreglo ya se encuentra ordenado. Pero este es en realidad el mejor caso solo para *Merge Sort* y *Merge-Insertion Sort*, porque por ejemplo para *Quick Sort*, es algo más cercano al peor caso que al mejor, ya que en 2 de sus implementaciones (primer elemento como pivote y último elemento como pivote), es de hecho, el peor caso.

Además, como se puede apreciar, el gráfico está hecho en escala logarítmica en base 10, lo que significa que hay que ser más cauteloso en este análisis.

Los principales puntos a analizar acerca de este gráfico son:

- Las 3 curvas inferiores del gráfico (azul, amarilla y celeste), correspondientes *Merge-Insertion Sort*, *Merge Sort* y *Quick Sort* mediante pivote aleatorio respectivamente, se encuentran bastante próximas entre sí y presentan una forma visual relativamente suave. A primera vista, debido a que el eje vertical se encuentra representado en escala logarítmica en base 10, las curvas parecen similares a funciones logarítmicas. Sin embargo, es importante tener presente que el tiempo real no está creciendo logarítmicamente, sino que la escala logarítmica comprime visualmente el crecimiento real de las funciones temporales.

Para aproximar experimentalmente el tipo de crecimiento, inspeccionamos dos puntos aproximados del gráfico:

$$P_1 \approx (n = 1500, t = 0,0001) \text{ y } P_2 \approx (n = 3000, t = 0,0002)$$

Se puede observar que al duplicar la cantidad de datos, el tiempo también aproximadamente se duplica. Esto sugiere experimentalmente un crecimiento muy cercano al lineal, lo cual resulta coherente con la teoría, ya que estos algoritmos, en escenarios favorables o promedio, poseen complejidad temporal de $O(n \log n)$, lo cual también es considerado como un crecimiento *quasilineal*.

En el caso particular de *Merge Sort* y *Merge-Insertion Sort*, este comportamiento resulta esperable, ya que ambos algoritmos dividen el arreglo de manera relativamente uniforme, evitando particiones extremadamente desbalanceadas. Además, en *Merge-Insertion Sort*, el uso de *Insertion Sort* sobre subarreglos pequeños reduce considerablemente el *overhead* recursivo, lo cual explica que experimentalmente su curva tienda a ubicarse ligeramente por debajo de la correspondiente a *Merge Sort*.

Por otro lado, el buen rendimiento experimental de *Quick Sort* con pivote aleatorio, se debe principalmente a que la selección aleatoria del pivote, reduce significativamente la probabilidad de generar particiones extremadamente desequilibradas de forma repetitiva, aproximando así el comportamiento promedio esperado de *Quick Sort*, el cual también corresponde a complejidad temporal $O(n \log n)$ bajo condiciones normales.

- Las otras 3 curvas del gráfico (verde, naranja y morada), correspondientes a las variantes de *Quick Sort* utilizando pivote al inicio, pivote al final y mediana de tres, respectivamente, se encuentran considerablemente más arriba que las anteriores y presentan un crecimiento claramente mayor. Visualmente, debido nuevamente a la escala logarítmica utilizada, las curvas siguen viéndose suaves; sin embargo, esto no implica que el crecimiento real sea logarítmico. De hecho, la escala logarítmica comprime fuertemente las diferencias de crecimiento, por lo que algoritmos con complejidad cuadrática pueden aparentar visualmente un crecimiento mucho más moderado.

Comparando experimentalmente las curvas inferiores con estas curvas superiores, se observa que el tiempo aumenta considerablemente más rápido a medida que crece el tamaño del arreglo. Esto resulta especialmente coherente con la teoría en las variantes que utilizan pivote al inicio y pivote al final.

En estos casos, el arreglo previamente ordenado ascendentemente produce sistemáticamente particiones extremadamente desequilibradas bajo el esquema de partición de Lomuto. Por ejemplo, si el pivote corresponde siempre al último elemento y el arreglo ya se encuentra ordenado, entonces dicho pivote será constantemente el mayor elemento del subarreglo, provocando divisiones del tipo $(n - 1, 0)$, en lugar de divisiones balanceadas o relativamente balanceadas.

Como consecuencia, la profundidad recursiva aumenta considerablemente y la recurrencia temporal se aproxima a $T(n) = T(n - 1) + O(n)$, cuya solución corresponde a una complejidad de $O(n^2)$, lo cual coincide completamente con el peor caso teórico de *Quick Sort* (el mejor caso genérico implementado).

En el caso de la variante mediante mediana de tres, teóricamente se esperaría un mejor rendimiento respecto a las variantes con pivote fijo, debido a que seleccionar la mediana entre el primer elemento, el central y el último reduce la probabilidad de generar pivotes extremadamente desfavorables. Sin

embargo, experimentalmente se observa que su curva permanece bastante próxima a las variantes con pivote al inicio y al final.

Lo anterior sugiere que, si bien la estrategia de mediana de tres logra evitar algunos pivotes particularmente malos, el arreglo previamente ordenado sigue produciendo particiones suficientemente desbalanceadas como para deteriorar considerablemente el rendimiento práctico del algoritmo. Además, el *overhead* adicional necesario para calcular la mediana en cada llamada recursiva también puede influir negativamente en el tiempo total de ejecución.

Por lo tanto, aunque la estrategia de mediana de tres suele mejorar el comportamiento promedio de *Quick Sort*, en este escenario experimental específico no logra separarse significativamente de las otras variantes de *Quick Sort* con pivote fijo, manteniendo un comportamiento empírico considerablemente menos eficiente que los algoritmos basados en *Merge Sort*.

- Normalmente, en el mejor caso real para cada algoritmo, esperaríamos que todos presenten complejidad temporal $O(n \log n)$, y que *Quick Sort*, especialmente en sus variantes mediante pivote aleatorio y mediana de tres, evidencie tiempos incluso ligeramente menores que *Merge Sort* y *Merge-Insertion Sort*. Esto se debe principalmente a que *Quick Sort* trabaja *in-place*, utilizando considerablemente menos memoria auxiliar que *Merge Sort* y *Merge-Insertion Sort*, los cuales requieren memoria adicional de orden $O(n)$ para realizar las fusiones. Además, *Quick Sort* suele presentar un menor *overhead* práctico asociado a movimientos de memoria y estructuras auxiliares. Sin embargo, al tratarse de una implementación del mejor caso genérica, basada en arreglos previamente ordenados, esto no fue lo que se observó experimentalmente.

4.6.1.2. Análisis de caso promedio

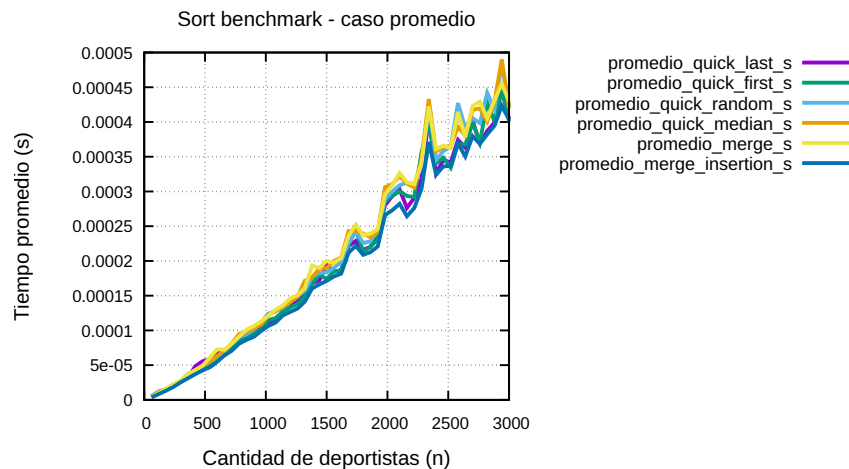


Figura 4.2: Benchmark de algoritmos de ordenamiento en caso promedio.

En este otro gráfico (que está en escala normal o lineal), se visualiza el comportamiento de los algoritmos de ordenamiento implementados en el caso promedio. En la implementación realizada, el caso promedio fue

uno genérico para todos los algoritmos, y consistió en generar un orden aleatorio para los elementos.

Los puntos principales a analizar acerca de este gráfico son:

- Se puede observar que todos los algoritmos presentan un crecimiento relativamente similar. Además, dicho crecimiento observado en todos, es ligeramente más pronunciado que uno lineal, pero no mucho más, lo cual concuerda con la teoría, ya que tanto *Merge Sort* como *Quick Sort*, y las variantes implementadas de cada uno de ellos, poseen una complejidad temporal promedio de $O(n \log n)$.
- A pesar de lo anterior, también se observan algunas pequeñas diferencias. Por ejemplo, la línea azul, correspondiente al algoritmo de *Merge-Insertion Sort*, presentó un comportamiento ligeramente mejor a los otros algoritmos, ya que su curva se ve ligeramente por debajo de las otras, lo que implica que tardó un poco menos en general. A pesar de ser una diferencia muy pequeña, igual el hecho de que sea notoria, significa que igual el hecho de combinar *Merge Sort* con *Insertion Sort*, ayuda a reducir un poquito el tiempo. Nosotros pensamos que puede deberse a que *Insertion Sort* evita acumular más *overhead* recursivo, lo que lo hace un poco más eficiente.

4.6.1.3. Análisis de peor caso

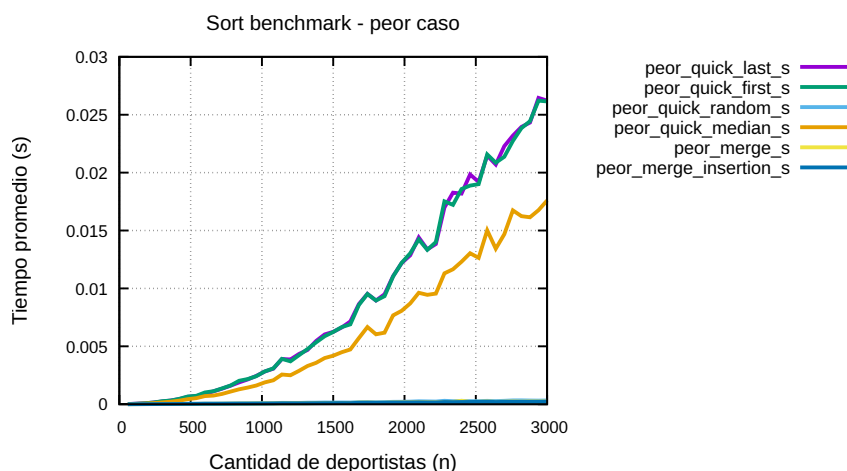


Figura 4.3: Benchmark de algoritmos de ordenamiento en peor caso.

Finalmente, este gráfico ilustra qué ocurre con los algoritmos de ordenamiento cuando se enfrentan al peor caso general definido para los experimentos, el cual consistió en utilizar arreglos ordenados en orden inverso. Cabe destacar que esta definición corresponde principalmente a un peor caso para *Quick Sort*, pero no necesariamente para *Merge Sort*.

Lo primero que se observa del gráfico es que está representado en escala lineal.

En base a ello, se puede analizar lo siguiente:

- Las variantes de *Quick Sort* que utilizan el primer o el último elemento como pivote fueron las que presentaron el peor rendimiento. Esto ocurre porque, bajo una entrada invertida, las particiones gene-

radas se vuelven extremadamente desbalanceadas, provocando que el algoritmo reduzca el problema de manera muy ineficiente en cada paso recursivo.

La forma de la curva se asemeja claramente a un crecimiento cuadrático, consistente con una complejidad temporal de $O(n^2)$, tal como lo establece la teoría para el peor caso de estas variantes.

- La variante de *Quick Sort* utilizando pivote por mediana de tres presenta un rendimiento mejor que las variantes anteriores, aunque sigue mostrando un crecimiento superior al de *Merge Sort*.

Esto puede explicarse porque seleccionar la mediana entre tres elementos permite evitar muchos casos de particiones extremadamente desbalanceadas, pero aun así no garantiza divisiones perfectamente equilibradas del arreglo en todos los casos.

- Por otro lado, *Merge Sort*, *Merge-Insertion Sort* y *Quick Sort* con pivote aleatorio muestran un rendimiento considerablemente mejor en este escenario.

En el caso de *Merge Sort* y *Merge-Insertion Sort*, esto se debe a que su complejidad temporal depende muy poco del orden inicial de los datos, manteniéndose cercana a $O(n \log n)$ en cualquier caso.

En cuanto a *Quick Sort* con pivote aleatorio, el rendimiento mejora porque la selección aleatoria del pivote reduce considerablemente la probabilidad de generar particiones extremadamente desbalanceadas de forma repetitiva.

4.6.1.4. Analisis Merge-Insertion Sort

En esta seccion, se analizara el rendimiento de Merge-Insertion Sort en el caso promedio con distintos threshholds, para determinar cual es el threshhold optimo para aplicar Insertion Sort en lugar de continuar con la division recursiva. Para esto, se compararan los tiempos obtenidos en el benchmark de Merge-Insertion Sort con distintos threshholds, y con Merge clasico y se analizara cual de ellos ofrece el mejor rendimiento en funcion del tamaño de entrada.

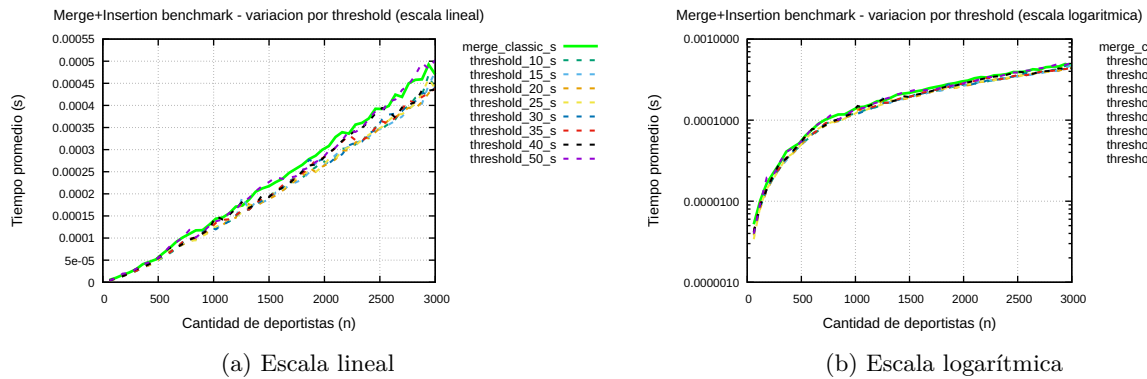


Figura 4.4: Evaluación de Merge-Insertion: comparación de thresholds en escala lineal y logarítmica.

Como se puede observar en ambos gráficos, el rendimiento de Merge-Insertion Sort varía dependiendo del threshhold utilizado para decidir cuándo aplicar Insertion Sort. En general, se observa que los threshholds más bajos (por ejemplo, 10 o 20) tienden a ofrecer un mejor rendimiento en comparación con threshholds más altos (como 50 o 100). Esto se debe a que Insertion Sort es particularmente eficiente para ordenar pequeños

subarreglos, y al aplicar esta estrategia con un treshhold bajo, se reduce el overhead recursivo asociado a dividir el arreglo en partes cada vez más pequeñas.

Esta diferencia se hace mas notoria al analizar el grafico en escala lineal, donde se puede apreciar claramente la superioridad en rendimiento de los treshholds mas bajos en comparacion a los mas altos y a la implementacion clasica de Merge Sort. Sin embargo, al analizar ambos gráficos en conjunto, se puede concluir que un treshhold de alrededor de 10 o 20 es óptimo para aplicar Insertion Sort dentro de Merge-Insertion Sort, ya que ofrece un rendimiento que puede no parecer significativo, pero para cantidades grandes de datos puede hacer la diferencia. Otra conclusion importantes es que el valor de treshhold exacto puede depender del tamaño de entrada y de la distribución de los datos, por lo que es recomendable realizar pruebas adicionales para determinar el treshhold más adecuado en cada caso específico.

4.6.1.5. Análisis de peor caso

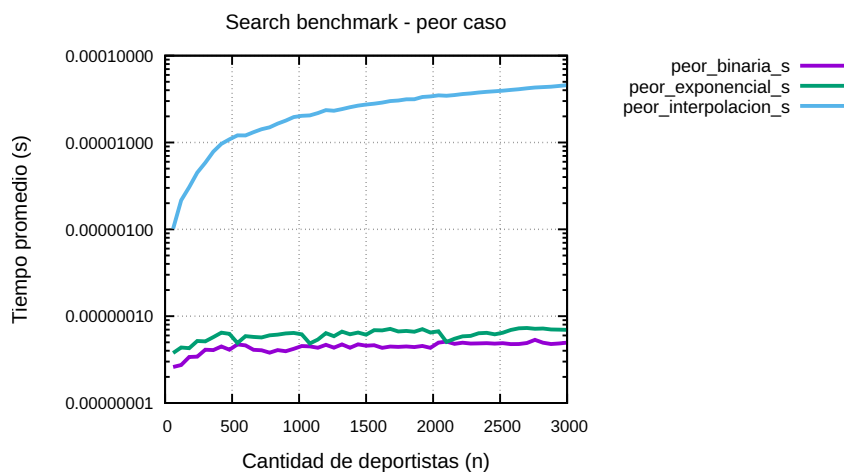


Figura 4.5: Benchmark de algoritmos de búsqueda en peor caso.

En este primer gráfico comparativo sobre los algoritmos de búsqueda, que está en escala logarítmica, se puede visualizar cómo se comportan los 3 algoritmos de búsqueda implementados, en el peor caso.

Aquí, el peor caso, fue implementado de forma un poquito más cercana a la realidad. Para búsqueda binaria y para búsqueda exponencial, el peor caso se implementó forzando a buscar el último ID, lo cual para la búsqueda binaria, sí entra dentro de un mal caso, ya que obliga a hacer todas las divisiones hasta llegar a la última posición, en tanto que para la búsqueda exponencial, entra dentro de algo aceptable como peor caso, aunque en realidad es más cercano a un caso promedio, ya que como se buscará el último ID siempre, si bien el algoritmo deberá recorrer secuencialmente todas los elementos del arreglo que estén en posiciones que sean potencias de 2 que estén dentro de los límites del *array*, luego de eso irá a la última posición y terminará sin pasar por la sección de búsqueda binaria. Por eso entra dentro de lo aceptable para un peor caso, pero hay casos peores, como por ejemplo que no esté exactamente al final, sino más bien dentro del último tramo.

En el caso de la búsqueda por interpolación, el peor caso se implementó de una forma personalizada respecto a los otros 2. Lo que se hizo fue buscar el penúltimo ID en vez del último, y modificar artificialmente el último ID para que sea muchísimo mayor al penúltimo ID, de modo que la búsqueda por interpolación deba avanzar a pasos chicos, lo cual es muy bueno para visualizar el peor caso.

Dicho esto, lo que se puede observar, es que los algoritmos de búsqueda binaria recursiva (representado por la línea morada) y búsqueda exponencial (representado por la línea de color verde oscuro), escalan increíblemente bien en este peor caso, porque su tiempo a medida que aumentan los datos crece de una forma muy lenta, que por la escala logarítmica ni siquiera se nota bien el tipo de crecimiento que es, pero que podemos deducir que es un crecimiento logarítmico en la realidad, tal y cual dice la teoría, ya que solo un crecimiento logarítmico, podría verse como un crecimiento tan lento en una escala que ya es logarítmica.

Por otro lado, también se puede observar que la búsqueda por interpolación, en su peor caso, tiene peor rendimiento en comparación a los otros 2 algoritmos en su otro peor caso. Basta con darse cuenta que el crecimiento nosotros lo vemos suave, similar a un crecimiento logarítmico, pero a diferencia de las curvas de los otros algoritmos, es un crecimiento que sí se nota. Además hay que recordar que estamos en escala logarítmica, lo cual siempre hace que un crecimiento que aparentemente se ve logarítmico, en realidad no lo sea.

De hecho si tomamos dos puntos aproximados sobre la curva de búsqueda por interpolación. Por ejemplo $P_1 \approx (n = 500, t = 0,00001)$, $P_2 \approx (n = 1000, t = 0,00002)$, notamos la tendencia de que duplicando los datos, se duplica el tiempo de búsqueda también en el peor caso, lo cual es consistente con una complejidad temporal de $O(n)$, y por lo tanto con la teoría que también dice lo mismo.

4.6.1.6. Análisis de caso promedio

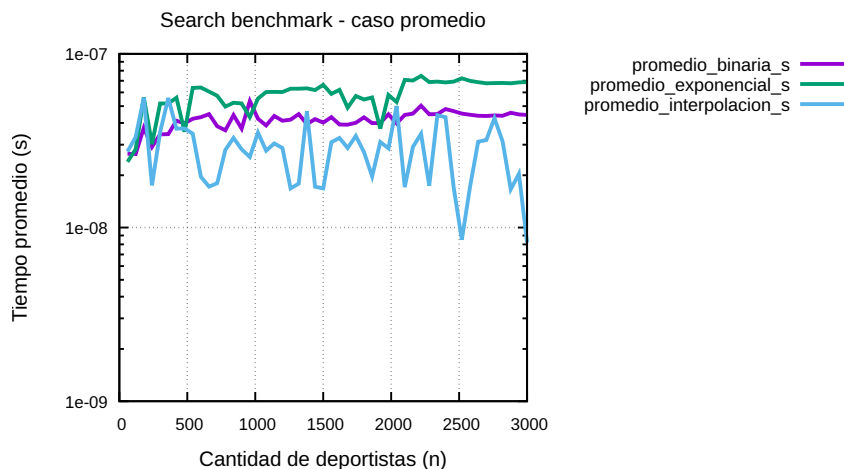


Figura 4.6: Benchmark de algoritmos de búsqueda en caso promedio.

En este gráfico se observa el comportamiento de los algoritmos de búsqueda en el caso promedio. Cabe destacar que el caso promedio implementado consistió en buscar un ID aleatorio dentro del rango existente

de IDs almacenados en el arreglo.

Este enfoque resulta relativamente representativo para búsqueda binaria y búsqueda exponencial, aunque debido a la naturaleza aleatoria de la selección, no siempre garantiza un comportamiento estrictamente promedio, ya que algunos IDs pueden favorecer o desfavorecer parcialmente el rendimiento dependiendo de su posición dentro del arreglo.

En el caso de la búsqueda por interpolación, ocurre una situación particular. Debido a que los IDs fueron generados de manera incremental y uniformemente distribuida, la fórmula de estimación de posición utilizada por *Interpolation Search* logra aproximar de manera muy precisa la ubicación del elemento buscado.

Por esta razón, independientemente del ID sorteado para la búsqueda, el algoritmo tiende experimentalmente a comportarse de manera muy cercana a un mejor caso, aproximándose incluso a un comportamiento constante en muchas ejecuciones.

Esto implica que el caso promedio implementado no representa completamente un caso promedio realista para búsqueda por interpolación. Sin embargo, modificar esta situación hubiese requerido alterar significativamente la distribución de los IDs generados, por lo que se decidió mantener la estructura original de los datos para los experimentos.

Los principales puntos a observar/analizar del gráfico son:

- El gráfico muestra una tendencia ligeramente ascendente para la búsqueda binaria y para la búsqueda exponencial, aunque con bastante ruido experimental, especialmente en la búsqueda exponencial. Aun así, ignorando parcialmente dicho ruido, se puede observar una tendencia coherente con la teoría, ya que el crecimiento es muy lento e incluso casi imperceptible, lo cual, teniendo en cuenta la escala logarítmica del gráfico, sugiere un comportamiento compatible con una complejidad temporal de $O(\log n)$, tal como lo establece la teoría para ambos algoritmos.
- La búsqueda por interpolación presenta una gran cantidad de ruido experimental e incluso una tendencia visual que aparenta ser más estable o ligeramente descendente en algunos tramos, lo cual resulta poco intuitivo. Sin embargo, esto puede explicarse porque el caso promedio implementado no representa realmente un caso promedio para este algoritmo, sino un escenario altamente favorable.
- Debido a la distribución incremental y uniforme de los IDs, la búsqueda por interpolación logra estimar de manera extremadamente precisa la posición del elemento buscado, provocando que experimentalmente el algoritmo se comporte de manera cercana a $O(1)$ en muchas ejecuciones.
- Además, los tiempos medidos son extremadamente pequeños, incluso del orden de los nanosegundos (10^{-9} s), por lo que diversos factores externos pueden influir significativamente en las mediciones experimentales, como variaciones del sistema operativo, carga del procesador, efectos de caché y otros procesos ejecutándose simultáneamente.

4.6.1.7. Análisis Búsqueda Binaria por Rangos

Para el análisis de búsqueda binaria por rangos, se comparo el rendimiento de la búsqueda para rangos promedios con el rendimiento de los peores. Como se puede observar en el grafico, el rendimiento de ambos casos es bastante similar, aunque el caso promedio presenta un rendimiento ligeramente mejor. Esto se debe a que en el caso promedio, al seleccionar un elemento aleatorio dentro del rango de puntajes, se obtiene una muestra representativa del comportamiento del algoritmo en condiciones promedio, lo que puede resultar en tiempos de búsqueda más rápidos en algunos casos. Por otro lado, el peor caso puede presentar tiempos de búsqueda más largos debido a la naturaleza específica de los datos o a la posición del elemento buscado dentro del rango. Sin embargo, en general, ambos casos muestran un rendimiento bastante cercano, lo que sugiere que la búsqueda binaria por rangos es eficiente tanto en situaciones promedio como en situaciones adversas.

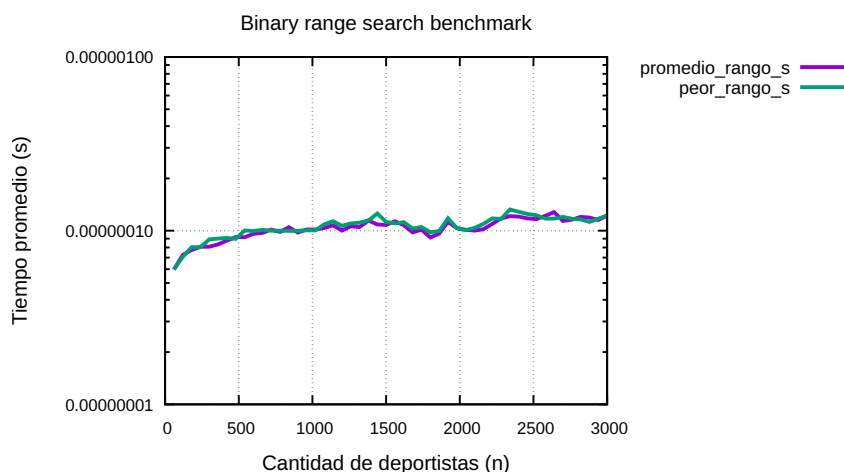


Figura 4.7: Benchmark de búsqueda binaria por rangos.

4.6.2. Análisis de Quick Select

En los gráficos de *Quick Select* se observa que el rendimiento del algoritmo depende de forma importante tanto del caso experimental construido como de la estrategia utilizada para seleccionar el pivote. En el caso promedio, las cuatro variantes presentan curvas muy cercanas entre sí y con un crecimiento gradual, lo que resulta coherente con el comportamiento esperado de *Quick Select*, ya que normalmente no necesita ordenar todo el arreglo, sino continuar solo por la partición donde se encuentra el elemento buscado. Finalmente, en el peor caso, se aprecia un aumento más notorio del tiempo de ejecución, especialmente en las variantes con pivote aleatorio y pivote en el primer elemento; sin embargo, los tiempos siguen siendo bajos en comparación con un ordenamiento completo. En conjunto, los resultados muestran que *Quick Select* es una alternativa adecuada para obtener el k -ésimo deportista según un criterio determinado, pero también evidencian que su rendimiento práctico puede variar según la elección del pivote y la forma en que quedan distribuidas las particiones durante la ejecución.

4.6.2.1. Análisis de mejor caso

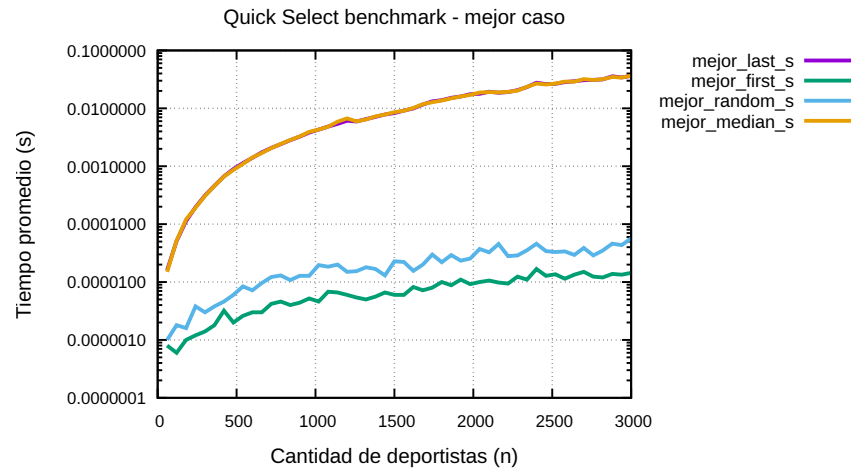


Figura 4.8: Benchmark de Quick Select en mejor caso.

En el mejor caso, las variantes con pivote en el primer elemento y pivote aleatorio presentan tiempos considerablemente menores, mientras que las variantes con último elemento y mediana de tres muestran un crecimiento mucho más alto; esto indica que, para esa disposición específica de los datos y de la posición buscada, algunas elecciones de pivote permiten acercarse rápidamente al elemento objetivo, mientras que otras generan particiones menos favorables.

4.6.2.2. Análisis de caso promedio

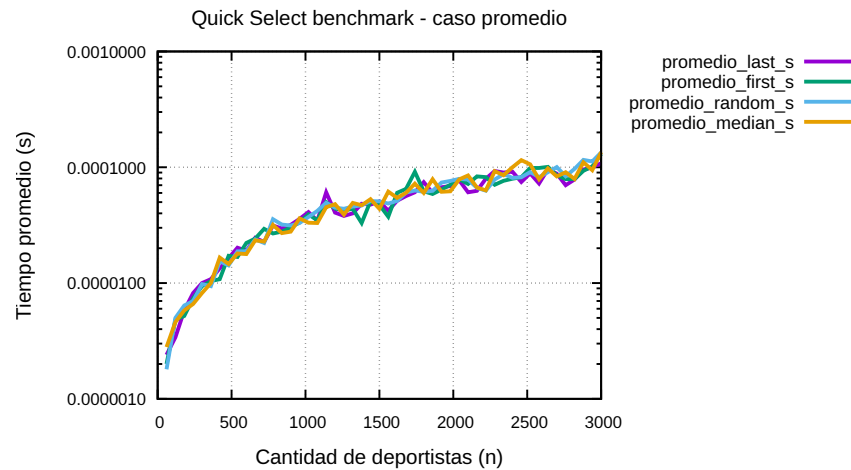


Figura 4.9: Benchmark de Quick Select en caso promedio.

En el caso promedio, las cuatro variantes presentan curvas muy cercanas entre sí y con un crecimiento gradual, lo que resulta coherente con el comportamiento esperado de *Quick Select*, ya que normalmente no necesita ordenar todo el arreglo, sino continuar solo por la partición donde se encuentra el elemento buscado.

4.6.2.3. Análisis de peor caso

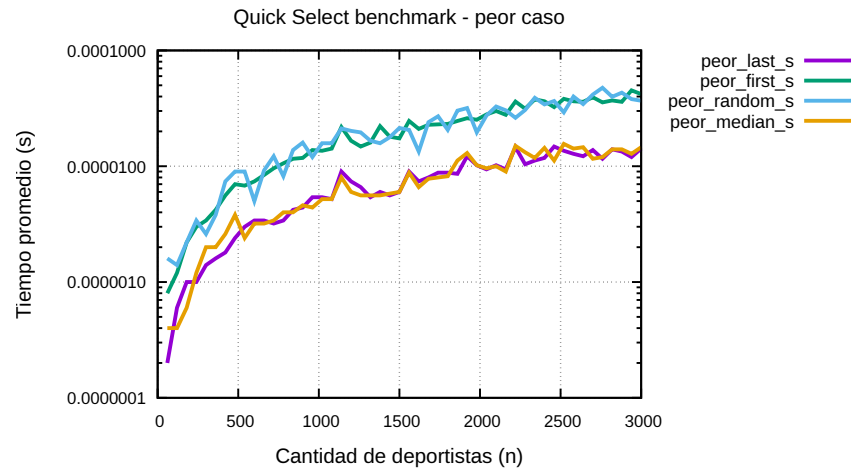


Figura 4.10: Benchmark de Quick Select en peor caso.

Finalmente, en el peor caso, se aprecia un aumento más notorio del tiempo de ejecución, especialmente en las variantes con pivote aleatorio y pivote en el primer elemento; sin embargo, los tiempos siguen siendo bajos en comparación con un ordenamiento completo.

En conjunto, los resultados muestran que *Quick Select* es una alternativa adecuada para obtener el k -ésimo deportista según un criterio determinado, pero también evidencian que su rendimiento práctico puede variar según la elección del pivote y la forma en que quedan distribuidas las particiones durante la ejecución.

4.6.3. Comparacion con algoritmos iterativos

En el proyecto anterior, se realizaron benchmark similares con respecto a los algoritmos de búsqueda y ordenamiento, pero utilizando implementaciones iterativas de cada uno de los algoritmos. Por lo tanto, en esta sección se comparará el rendimiento de los algoritmos recursivos implementados en este proyecto, con el rendimiento de los algoritmos iterativos implementados en el proyecto anterior. Para esto, se compararán los tiempos obtenidos en los benchmarks de búsqueda y ordenamiento para ambos tipos de algoritmos, y se analizará cuál de ellos ofrece un mejor rendimiento en función del mismo tamaño de entrada.

4.6.3.1. Comparacion de algoritmos de Búsquedas

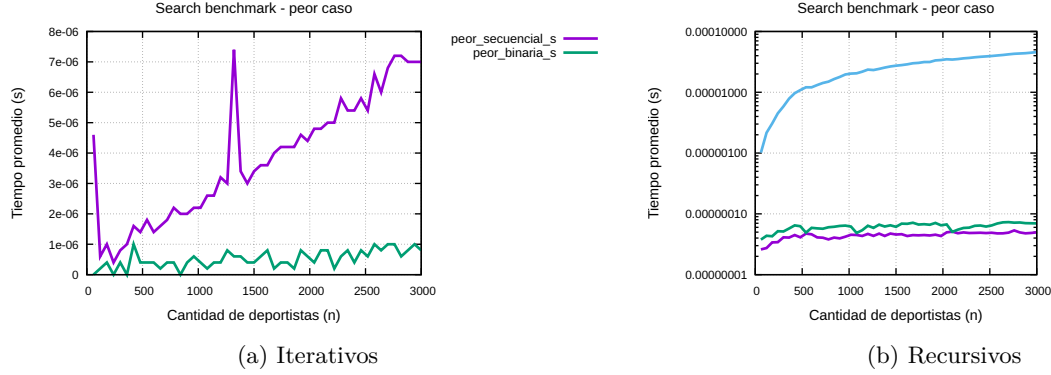


Figura 4.11: Benchmark de algoritmos de búsqueda: comparativa iterativos vs recursivos.

Al analizar los graficos, las conclusiones son claras. La mayoría de algoritmos recursivos (a excepción de la búsqueda por interpolacion), presentan un rendimiento mejor que sus equivalentes iterativos, lo cual se puede observar claramente al comparar los tiempos de ambos graficos. Esto se debe a que las implementaciones recursivas presentan tiempos teoricos de ejecucion menores que las implementaciones iterativas, lo cual se refleja en los tiempos medidos en los benchmarks. Por ejemplo, la búsqueda binaria recursiva tiene una complejidad temporal de $O(\log n)$, mientras que la búsqueda secuencial tiene una complejidad temporal de $O(n)$, lo cual explica la diferencia de rendimiento observada en el gráfico. Sin embargo, es importante destacar que la diferencia entre Binaria iterativo y binaria recursivo no es tan grande, lo cual se debe a que ambos algoritmos tienen la misma complejidad temporal de $O(\log n)$, aunque la implementación recursiva puede tener un overhead adicional debido a las llamadas recursivas, lo cual puede afectar ligeramente el rendimiento en algunos casos. En el caso de la búsqueda por interpolacion, su rendimiento es peor que el de los otros algoritmos, lo cual se debe a que su complejidad temporal en el peor caso es de $O(n)$, lo cual es significativamente peor que la complejidad temporal de los otros algoritmos de búsqueda implementados. Por lo tanto, aunque la búsqueda por interpolación puede ser eficiente en ciertos casos específicos, su rendimiento general es inferior al de los otros algoritmos de búsqueda implementados.

4.6.3.2. Comparacion de algoritmos de Ordenamiento

En esta seccion se compararan las implementaciones iterativas de los algoritmos de ordenamiento realizados en el proyecto anterior, con las implementaciones recursivas realizadas en este proyecto. Para esto, se compararan los tiempos obtenidos en los benchmarks de ordenamiento para ambos tipos de algoritmos, y se analizara cual de ellos ofrece un mejor rendimiento en funcion del mismo tamaño de entrada.

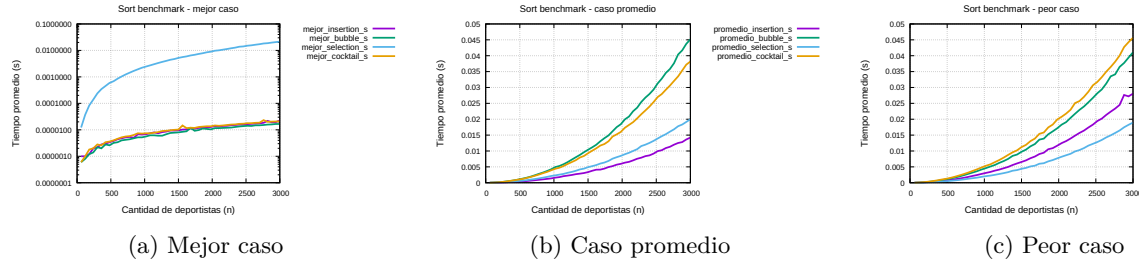


Figura 4.12: Benchmarks de ordenamiento secuencial: mejor, promedio y peor caso.

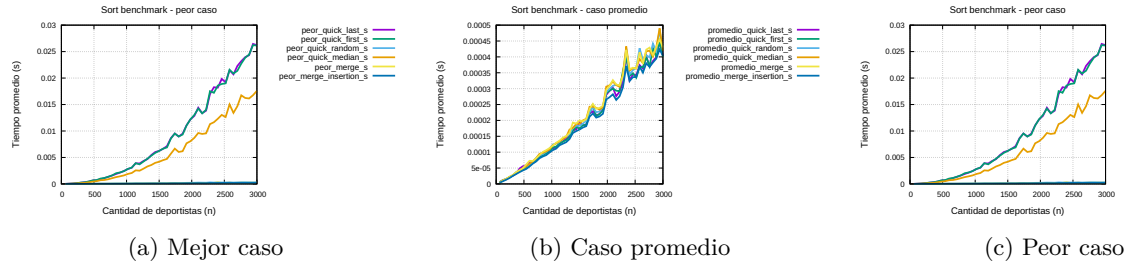


Figura 4.13: Benchmarks de ordenamiento recursivo: mejor, promedio y peor caso.

Al comparar los algoritmos de ordenamiento iterativos con sus equivalentes recursivos, se observa que ambos grupos conservan el mismo comportamiento asintótico esperado, pero difieren en el costo constante asociado a su implementación. En el caso de los algoritmos recursivos, la estructura de llamadas facilita la división natural del problema y suele producir un beneficio conceptual el cual introduce un *overhead* adicional por las llamadas recursivas y por el manejo de la pila de ejecución.

Desde el punto de vista experimental, los ordenamientos iterativos tienden a ser ligeramente más estables cuando se trabaja con entradas grandes y con muchas repeticiones del benchmark, porque evitan parte del costo de la recursión y reducen la presión sobre la pila. Esto se nota con mayor claridad en algoritmos como *Quick Sort*, donde la elección del pivote y el comportamiento de las particiones influyen en el rendimiento final. En cambio, en *Merge Sort* y *Merge-Insertion Sort*, la versión recursiva no se queda atrás en cuanto a rendimiento, ya que la división uniforme del arreglo favorece su funcionamiento (ausencia de overhead) y la diferencia con la versión iterativa no siempre resulta significativa.

En términos generales, puede afirmarse que las versiones recursivas son más naturales para expresar algoritmos de tipo **Divide y Vencerás**, mientras que las versiones iterativas pueden ofrecer una ligera ventaja práctica cuando se busca minimizar el costo de ejecución y el uso de pila. Por ello, la elección entre una u otra implementación depende del objetivo del trabajo: si se prioriza claridad y relación directa con la teoría, la versión recursiva es más conveniente; si se prioriza control fino del rendimiento, la versión iterativa puede ser una alternativa atractiva.

Por ultimo, no esta demas mencionar que los algoritmos como burbuja, insertion y selection, aunque son algoritmos de ordenamiento clásicos y fáciles de entender, presentan un rendimiento muy inferior al de los algoritmos basados en **Divide y Vencerás**, como *Merge Sort* y *Quick Sort*, especialmente a medida que el tamaño de la entrada aumenta. Por lo tanto, aunque pueden ser útiles para ordenar pequeñas cantidades de datos, queda evidenciado que no son recomendables para aplicaciones prácticas que requieran eficiencia en el ordenamiento de grandes conjuntos de datos.

4.6.4. Análisis General

En términos generales, los resultados experimentales permiten observar que los algoritmos recursivos implementados en este proyecto presentan un rendimiento claramente superior frente a los algoritmos iterativos trabajados en la etapa anterior, especialmente en los problemas de ordenamiento. Mientras los algoritmos iterativos clásicos, como Bubble Sort, Selection Sort, Insertion Sort y Cocktail Shaker Sort, tienden a presentar comportamientos cuadráticos en gran parte de los escenarios, los algoritmos basados en *Divide y Vencerás*, como *Merge Sort*, *Merge-Insertion Sort* y varias variantes de *Quick Sort*, logran mantener un crecimiento mucho más controlado, cercano a $O(n \log n)$ en condiciones favorables o promedio. En búsqueda ocurre una situación similar: los métodos que reducen progresivamente el espacio de búsqueda, como búsqueda binaria recursiva, búsqueda exponencial y búsqueda por interpolación en condiciones adecuadas, resultan más eficientes que una revisión secuencial directa de los datos. No obstante, también se observa que los resultados experimentales no siempre coinciden de manera exacta con la teoría, lo cual es esperable, ya que las mediciones dependen de factores como el hardware utilizado, la carga del sistema operativo, la caché del procesador, el compilador, el tamaño de entrada, la forma en que se construyen los casos de prueba y el costo constante de cada implementación. Por ello, la complejidad asintótica permite explicar la tendencia general, pero no determina por sí sola todos los tiempos reales medidos. Considerando el conjunto de datos utilizado en este proyecto, su forma de generación, la distribución incremental de los ID y los resultados obtenidos en los benchmarks, se concluye que la alternativa más conveniente para el ordenamiento es *Merge-Insertion Sort* utilizando un *threshold* bajo, mientras que para la búsqueda por ID la opción más favorable es la búsqueda por interpolación, debido a que la distribución uniforme de los identificadores permite estimar con alta precisión la posición del elemento buscado.

CONCLUSIÓN

El desarrollo de este proyecto permitió ampliar el sistema de gestión de deportistas construido previamente, incorporando algoritmos basados en la estrategia de *Divide y Vencerás* para resolver problemas de ordenamiento, búsqueda y selección. A diferencia del trabajo anterior, centrado principalmente en algoritmos iterativos clásicos, esta segunda etapa permitió analizar técnicas recursivas más eficientes y observar cómo su comportamiento depende tanto de la complejidad teórica como de las decisiones concretas de implementación.

En relación con los algoritmos de ordenamiento, la implementación de *Merge Sort* permitió contar con un método estable en cuanto a complejidad temporal, ya que mantiene un comportamiento $O(n \log n)$ en mejor caso, caso promedio y peor caso. Esto se debe a que el algoritmo divide el arreglo de manera relativamente uniforme y luego combina los subarreglos mediante un proceso de mezcla lineal. En la práctica, esta propiedad se reflejó en un rendimiento consistente frente a distintos escenarios de entrada, lo que confirma su utilidad cuando se requiere un comportamiento predecible. La variante *Merge-Insertion Sort* permitió estudiar el efecto de combinar dos estrategias de ordenamiento. El uso de *Insertion Sort* en subarreglos pequeños busca reducir el costo asociado a llamadas recursivas demasiado finas, aprovechando que dicho algoritmo suele comportarse bien en entradas de tamaño reducido. Por ello, esta variante resulta relevante desde un punto de vista experimental: aunque mantiene la misma complejidad asintótica que *Merge Sort*, puede presentar mejoras prácticas dependiendo del umbral utilizado. Esto demuestra que dos algoritmos con igual complejidad teórica pueden diferir en rendimiento real debido a constantes ocultas, uso de memoria y sobrecarga de implementación.

En el caso de *Quick Sort*, los resultados permiten evidenciar con claridad la importancia de la selección del pivote. Las variantes que utilizan el primer o el último elemento como pivote pueden degradarse significativamente cuando la entrada ya se encuentra ordenada o invertida, debido a que generan particiones altamente desbalanceadas. Este comportamiento coincide con el peor caso teórico de *Quick Sort*, cuya complejidad puede alcanzar $O(n^2)$. En cambio, el pivote aleatorio y la mediana de tres buscan reducir la probabilidad de particiones desfavorables, acercando el comportamiento del algoritmo a su caso promedio $O(n \log n)$. De esta forma, el proyecto muestra que *Quick Sort* no depende únicamente de su estructura general, sino también de una decisión aparentemente simple como la elección del pivote.

En relación con los algoritmos de búsqueda, los resultados experimentales revelaron diferencias significativas entre las distintas estrategias implementadas. La búsqueda binaria recursiva presentó tiempos consistentemente bajos, confirmando su complejidad teórica $O(\log n)$, con un crecimiento prácticamente imperceptible al aumentar el tamaño de entrada. La búsqueda exponencial, aunque mantiene la misma complejidad asintótica, mostró un comportamiento ligeramente superior en términos de tiempos observados, lo que sugiere que las constantes ocultas asociadas a su fase inicial de expansión impactan el rendimiento en la práctica. Por su parte, la búsqueda por interpolación demostró una dependencia crítica respecto a la distribución de los datos: en entradas uniformemente distribuidas alcanzó rendimientos comparables a la búsqueda binaria, pero degradó notablemente cuando los datos presentaban sesgos o agrupamientos, confirmando la importancia de considerar la naturaleza específica del conjunto de datos.

La búsqueda binaria por rangos contribuyó con una funcionalidad práctica al sistema, permitiendo obtener subconjuntos de deportistas dentro de un intervalo de puntajes determinado. Los benchmarks mostraron que esta variante mantiene un comportamiento eficiente incluso al expandir el rango de búsqueda, debido a que realiza dos búsquedas binarias para localizar los límites. La implementación resultó útil para ampliar las capacidades del sistema más allá de búsquedas exactas, facilitando consultas de rango que son comunes en aplicaciones de gestión de datos.

La implementación de *Quick Select* permitió resolver el problema de encontrar el k -ésimo mejor deportista sin ordenar completamente el arreglo. Esta diferencia es importante, ya que ordenar todos los datos puede ser innecesario cuando solo se necesita conocer una posición específica. Al reutilizar la lógica de partición de *Quick Sort*, *Quick Select* permite descartar una parte del arreglo en cada paso recursivo y concentrarse únicamente en el segmento donde puede encontrarse el elemento buscado. En promedio, esto permite alcanzar complejidad $O(n)$, aunque el algoritmo también puede degradarse a $O(n^2)$ si las particiones son muy desbalanceadas.

Desde el punto de vista experimental, los *benchmarks* realizados permitieron contrastar la teoría con el comportamiento real de los algoritmos. Las gráficas generadas a partir de los archivos CSV facilitaron observar tendencias de crecimiento, diferencias entre variantes y efectos de los casos de prueba. En particular, los resultados de ordenamiento mostraron que *Merge Sort* y *Merge-Insertion Sort* poseen un rendimiento más estable frente a distintas entradas, mientras que *Quick Sort* puede variar considerablemente según la variante de pivote utilizada. En las búsquedas, los algoritmos basados en división del espacio de búsqueda mantuvieron tiempos reducidos frente al crecimiento de n , mientras que la búsqueda por interpolación dependió con mayor fuerza de la distribución de los datos.

En conclusión, el proyecto permitió comprobar que la estrategia de *Divide y Vencerás* ofrece herramientas eficientes para trabajar con conjuntos de datos crecientes, especialmente cuando se compara con enfoques iterativos más simples. Sin embargo, también se evidenció que la complejidad asintótica no explica por sí sola todo el comportamiento práctico de un algoritmo. Factores como la elección del pivote, el uso de memoria auxiliar, el umbral de cambio entre algoritmos, la distribución de los datos y el diseño de los casos experimentales influyen directamente en los tiempos observados. Por ello, la selección de un algoritmo adecuado debe considerar tanto su fundamento teórico como el contexto concreto en que será aplicado.

ANEXOS

6.1. Código fuente

Repositorio de GitHub: https://github.com/ayrtonmo/tarea_algoritmos_2

6.2. Contribución por integrante

6.2.1. Iván Gallardo

- Implementación de Búsqueda exponencial por ID.
- Quick Sort con sus pivotes.
- Quick Select.
- Arreglo de error en Búsqueda por rangos que impedía su correcto funcionamiento.
- Graficación de Quick Select y Búsqueda Binaria por rangos.
- Benchmark para Búsqueda binaria por rangos.
- Marco teórico y conclusión del informe.
- Análisis teórico y empírico del informe.

6.2.2. Pablo Gómez

- Versión numérica de Búsqueda Exponencial.
- Búsqueda por interpolación.
- Benchmark para Quick Select.
- Resumen e introducción del informe.
- Análisis teórico y empírico del informe.

6.2.3. Ayrton Morrison

- Limpieza de repositorio anterior.
- Merge Sort y su optimización.
- Búsqueda binaria recursiva y búsqueda binaria por rangos.
- Mejora de legibilidad del código.
- Actualizaciones y adaptaciones de los benchmarks.
- Mejora al código para gráficos más interpretables.

- Benchmark de merge sort con distintos thresholds.
- Avance de sección Benchmark del informe.
- Análisis teórico y empírico del informe.